

■ Python Program Collection

Rewritten Edition

*All programs perform the same operations as the originals
but are rewritten with fresh variable names, alternative logic,
and an improved layout for clarity.*

Table of Contents

1. Basic Programs & Variables
2. Scope — Local, Global & Nonlocal
3. Strings & Type Conversion
4. Conditions & Loops
5. Functions
6. Patterns & Star Programs
7. Recursion & Algorithms
8. Lists
9. Dictionaries & Tuples
10. File Handling
11. NumPy
12. Pandas & Data Analysis
13. Matplotlib — Graphs & Charts

1. Basic Programs & Variables

■ add_numbers.py

↘ Reads two integers and prints their sum.

```
a = int(input('Enter the first number : '))
b = int(input('Enter the second number: '))
print('Sum =', a + b)
```

■ sum_series.py

↘ Computes $1+2+\dots+n$ using a for-loop accumulator.

```
n = int(input('Enter upper limit: '))
acc = 0
for i in range(1, n + 1):
    acc += i
print('Sum of series =', acc)
```

■ momentum.py

↘ Calculates linear momentum ($p = m \times v$).

```
m = float(input('Mass (kg) : '))
v = float(input('Velocity (m/s): '))
print('Momentum =', m * v, 'kg·m/s')
```

■ age_currency.py

↘ Converts birth year → age and salary Rs → USD.

```
birth_yr = int(input('Birth year : '))
salary = float(input('Salary (Rs): '))
age = 2025 - birth_yr
usd = salary / 83
print('Age:', age)
print('Salary in USD: {:.2f}'.format(usd))
print('===== END =====')
```

■ kinetic_energy.py

↘ Calculates kinetic energy ($KE = \frac{1}{2}mv^2$).

```
m = float(input('Mass (kg) : '))
v = float(input('Velocity (m/s): '))
ke = 0.5 * m * v ** 2
print('Kinetic Energy =', ke, 'J')
```

■ find_largest.py

↘ Finds the largest among five user-supplied numbers.

```
nums = [int(input(f'Enter number {i+1}: ')) for i in range(5)]
print('Largest number:', max(nums))
```

■ while_counter.py

↘ Counts 1 to 5 using a while loop.

```
counter = 1
while counter <= 5:
    print(counter)
    counter += 1
```

■ data_types.py

🔗 *Demonstrates Python's built-in data types.*

```
qty      = 15
rate     = 8.5
label    = 'Python'
is_done  = False
for val in (qty, rate, label, is_done):
    print(type(val))
```

■ step_range.py

🔗 *Prints values from 15 down to 3 stepping by -3.*

```
for x in range(15, 2, -3):
    print(x)
```

2. Scope — Local, Global & Nonlocal

■ `local_scope.py`

↘ Local variable accessible only inside the function.

```
def show():
    inner = 25
    print(inner)

show()
# print(inner) ← NameError: inner is not defined here
```

■ `global_scope.py`

↘ Reads and modifies a global variable from inside a function.

```
score = 100

def display():
    print('Score:', score)

display()
print('Score:', score)

score = 30

def update():
    global score
    score = 60

update()
print('Updated Score:', score)
```

■ `scope_demo.py`

↘ Shows how the same name resolves differently at each scope level.

```

x = 7

def first():
    x = 14
    print('Inside first():', x)

first()
print('Global x:', x)
print('---')

x = 40

def second():
    print('Inside second():', x)

second()
print('Global x:', x)
print('---')

x = 12

def third():
    global x
    x = 22

third()
print('Global x after third():', x)

```

■ `nonlocal_scope.py`

↘ *Demonstrates the `nonlocal` keyword in nested functions.*

```

def outer():
    value = 300
    def inner():
        nonlocal value
        value = 500
    inner()
    print('Outer value after inner():', value)

outer()

```

■ `nonlocal2.py`

↘ *Another `nonlocal` example with parent/child terminology.*

```
def parent():  
    total = 15  
    def child():  
        nonlocal total  
        total = 35  
        print('Child total:', total)  
    child()  
    print('Parent total:', total)  
  
parent()
```

3. Strings & Type Conversion

■ salary_stats.py

🔗 Collects monthly salaries and prints total, average, max, min.

```
months = int(input('How many months? '))
salaries = []
for m in range(1, months + 1):
    amt = int(input(f'  Salary for month {m}: '))
    salaries.append(amt)

print('Total    :', sum(salaries))
print('Average :', sum(salaries) / months)
print('Highest :', max(salaries))
print('Lowest  :', min(salaries))
```

■ word_splitter.py

🔗 Splits a sentence into words; splits marks by comma; counts words.

```
sentence = input('Enter a sentence: ')
print('Word list :', sentence.split())

raw_marks = input('Marks (comma-separated): ')
print('Marks list:', raw_marks.split(','))

line = input('Enter a sentence: ')
print('Word count:', len(line.split()))
```

■ type_cast.py

🔗 Demonstrates implicit int-to-float type conversion.

```
n = 20
print('Before cast:', type(n))
n = float(n)
print('After cast :', type(n))
```


4. Conditions & Loops

■ times_table.py

\ Prints the multiplication table (1–10) for a given number.

```
num = int(input('Enter a number: '))
for multiplier in range(1, 11):
    print(f'{num} x {multiplier} = {num * multiplier}')
```

■ leap_year.py

\ Checks whether a given year is a leap year.

```
yr = int(input('Enter year: '))
if (yr % 4 == 0 and yr % 100 != 0) or (yr % 400 == 0):
    print(yr, 'is a Leap Year')
else:
    print(yr, 'is NOT a Leap Year')
```

■ sum_and_check.py

\ Sums 1..n, then checks if a number is divisible by both 3 and 5.

```
limit = int(input('Enter limit: '))
total = sum(range(1, limit + 1))
print('Sum 1 to', limit, '=', total)
print('-----')
n = int(input('Enter a number: '))
if n % 3 == 0 and n % 5 == 0:
    print(n, 'is divisible by both 3 and 5')
else:
    print(n, 'is NOT divisible by both 3 and 5')
```

5. Functions

■ is_even_lambda.py

↘ Lambda function that returns True if a number is even.

```
is_even = lambda n: n % 2 == 0
print(is_even(10))    # True
print(is_even(7))     # False
```

■ even_odd_sums.py

↘ Returns the sum of even and odd numbers in a given range.

```
def split_sums(lo, hi):
    even_s = odd_s = 0
    for n in range(lo, hi + 1):
        if n % 2 == 0:
            even_s += n
        else:
            odd_s += n
    return even_s, odd_s

start = int(input('Start: '))
end = int(input('End : '))
ev, od = split_sums(start, end)
print('Even Sum:', ev)
print('Odd Sum:', od)
```

■ bonus_calc.py

↘ Calculates employee bonus based on salary, experience and rating.

```
def calc_bonus(pay, yrs, rating):
    if pay >= 50000 and yrs >= 5 and rating >= 4:
        print('High Bonus =', pay * 0.20)
    elif pay >= 30000 and yrs >= 3 and rating >= 3:
        print('Medium Bonus =', pay * 0.10)
    elif pay >= 20000 and yrs >= 1 and rating >= 2:
        print('Low Bonus =', pay * 0.05)
    else:
        print('No Bonus')

calc_bonus(
    float(input('Salary : ')),
    int(input('Years : ')),
    int(input('Rating : '))
)
```

■ scholarship.py

↘ Determines scholarship eligibility from marks, income, attendance.

```
def check_scholarship(marks, income, attend):
    if marks >= 85 and income <= 200000 and attend >= 90:
        print('Full Scholarship')
    elif marks >= 75 and income <= 300000 and attend >= 80:
        print('Partial Scholarship')
    elif marks >= 60 and income <= 400000 and attend >= 75:
        print('Financial Assistance Available')
    else:
        print('Not Eligible')

check_scholarship(
    int(input('Marks      : ')),
    int(input('Income     : ')),
    int(input('Attendance: '))
)
```

6. Patterns & Star Programs

■ patterns.py

🔪 Six classic patterns: ascending/descending stars, numbered rows, repeated digits, pyramid, filled rectangle.

```
# Pattern 1 - ascending stars
for r in range(1, 6):
    print('*' * r)

# Pattern 2 - descending stars
for r in range(5, 0, -1):
    print('*' * r)

# Pattern 3 - numbered triangle
for r in range(1, 6):
    print(''.join(str(c) for c in range(1, r + 1)))

# Pattern 4 - repeated digit
for r in range(1, 6):
    print(str(r) * r)

# Pattern 5 - centred pyramid
sz = 4
for r in range(sz):
    print(' ' * (sz - r - 1) + '*' * (2 * r + 1))

# Pattern 6 - filled rectangle
for _ in range(5):
    print('*' * 5)
```

7. Recursion & Algorithms

■ fibonacci.py

↘ Recursive Fibonacci sequence up to n terms.

```
def fib(n):
    if n == 0: return 0
    if n == 1: return 1
    return fib(n - 1) + fib(n - 2)

terms = int(input('Number of terms: '))
print(' '.join(str(fib(i)) for i in range(terms)))
```

■ factorial.py

↘ Iterative factorial using math.prod (Python 3.8+).

```
import math

n = int(input('Enter a number: '))
print('Factorial =', math.factorial(n))
```

■ reverse_number.py

↘ Reverses a positive integer without converting to string.

```
num = int(input('Enter number: '))
rev = 0
while num > 0:
    rev = rev * 10 + num % 10
    num //= 10
print('Reversed:', rev)
```

■ prime_range.py

↘ Lists prime numbers in a user-defined range using a helper.

```
def is_prime(n):
    if n < 2: return False
    return all(n % d != 0 for d in range(2, n))

lo = int(input('Start: '))
hi = int(input('End : '))
primes = [n for n in range(lo, hi + 1) if is_prime(n)]
print('Primes:', *primes)
```

■ mult_table.py

↘ Prints a formatted multiplication table for any number.

```
def print_table(n):
    for i in range(1, 11):
        print(f'{n} × {i:2d} = {n * i}')

print_table(int(input('Enter number: ')))
```

8. Lists

■ list_operations.py

🔗 Demonstrates append, insert, remove, pop, sort, reverse and membership test.

```
nums = [5, 10, 15, 20]
print('Original :', nums)
print('Index 0  :', nums[0])

nums[1] = 12
print('After update :', nums)

nums.append(25)      ; print('Append 25  :', nums)
nums.insert(2, 8)    ; print('Insert 8@2 :', nums)
nums.remove(20)      ; print('Remove 20  :', nums)
nums.pop(0)          ; print('Pop idx 0  :', nums)
print('Length :', len(nums))

nums.sort()          ; print('Sorted   :', nums)
nums.reverse()        ; print('Reversed:', nums)
print('12 in list?', 12 in nums)

print('Elements:', *nums)
```

■ list_extra.py

🔗 More list operations on a second list.

```
data = [11, 22, 33, 44, 55]
print('Original:', data)
data.append(66)    ; print('Append 66 :', data)
data.insert(2, 27) ; print('Insert 27 :', data)
data.remove(44)    ; print('Remove 44 :', data)
print('Length:', len(data))
data.sort()        ; print('Sorted   :', data)
```

9. Dictionaries & Tuples

■ dict_ops.py

🔗 *CRUD operations on a student dictionary.*

```
student = {'name': 'Aman', 'age': 19, 'course': 'CS'}
print('Original      :', student)

student['city'] = 'Mumbai'
print('After add city :', student)

student['age'] = 20
print('After age edit :', student)

student.pop('course')
print('After pop course:', student)

print('Name:', student['name'])
```

■ tuple_ops.py

🔗 *Indexing, slicing, concatenation, repetition and iteration on tuples.*

```
pts = (5, 10, 15, 20, 25)
print('Tuple   :', pts)
print('First   :', pts[0])
print('Last    :', pts[-1])
print('Slice   :', pts[:3])
print('Length  :', len(pts))

extra = (30, 35)
print('Concat  :', pts + extra)
print('Repeat  :', pts * 2)
print('15 in?   :', 15 in pts)

for v in pts:
    print(v, end=' ')
print()
print('Max:', max(pts), ' Min:', min(pts))
```

10. File Handling

■ file_rw.py

🔗 Write, append, and read student data using a context manager.

```
name = input('Student name : ')
marks = input('Marks          : ')

# Write
with open('record.txt', 'w') as f:
    f.write('Student Information\n')
    f.write('-----\n')
    f.write(f'Name : {name}\n')
    f.write(f'Marks: {marks}\n')
print('Data written successfully.')

# Append
grade = input('Grade: ')
with open('record.txt', 'a') as f:
    f.write(f'Grade: {grade}\n')
print('Data appended successfully.')

# Read
with open('record.txt', 'r') as f:
    print('\n--- File Contents ---')
    print(f.read())
```


11. NumPy

■ numpy_arrays.py

🔗 Creates two arrays and prints them.

```
import numpy as np

arr_a = np.array([5, 10, 15])
arr_b = np.array([20, 25, 30])
print('Array A:', arr_a)
print('Array B:', arr_b)
print('A + B :', arr_a + arr_b)
```

■ numpy_matrix.py

🔗 Creates a 1-D array and a 3×2 zero matrix.

```
import numpy as np

vec = np.array([2, 4, 6, 8])
print('Vector:', vec)

zeros = np.zeros((3, 2))
print('3×2 Zero Matrix:')
print(zeros)
```

12. Pandas & Data Analysis

■ pandas_analysis.py

🔪 Creates a student DataFrame; prints head/tail, statistics, filtered rows, sorting, and column add/drop.

```
import pandas as pd

data = {
    'Name'   : ['Riya', 'Kunal', 'Meena', 'Ajay', 'Neha'],
    'Age'    : [21, 22, 20, 23, 21],
    'Marks'  : [88, 75, 91, 84, 79],
    'City'   : ['Pune', 'Delhi', 'Mumbai', 'Pune', 'Nagpur'],
}
df = pd.DataFrame(data)

print('Full DataFrame:\n', df)
print('\nFirst 3 rows:\n', df.head(3))
print('\nLast 2 rows:\n', df.tail(2))
print('\nStatistics:\n', df.describe())
print('\nNames:\n', df['Name'])
print('\nName & Marks:\n', df[['Name', 'Marks']])
print('\nMarks > 80:\n', df[df['Marks'] > 80])

df['Status'] = 'Pass'
print('\nWith Status:\n', df)
df.drop('Status', axis=1, inplace=True)
print('\nSorted by Marks:\n', df.sort_values('Marks'))
```

13. Matplotlib — Graphs & Charts

■ charts.py

🔗 *Plots line, bar and pie charts for monthly profit data.*

```
import matplotlib.pyplot as plt

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May']
profits = [120, 180, 220, 200, 260]

plt.figure(figsize=(6, 4))
plt.plot(months, profits, marker='o', color='purple')
plt.title('Monthly Profit – Line Graph')
plt.xlabel('Month') ; plt.ylabel('Profit')
plt.tight_layout() ; plt.show()

plt.figure(figsize=(6, 4))
plt.bar(months, profits, color='violet')
plt.title('Monthly Profit – Bar Graph')
plt.xlabel('Month') ; plt.ylabel('Profit')
plt.tight_layout() ; plt.show()

plt.figure(figsize=(5, 5))
plt.pie(profits, labels=months, autopct='%1.1f%%',
        startangle=90)
plt.title('Profit Distribution')
plt.show()
```

■ multi_charts.py

🔗 *Line, bar, pie, scatter, histogram and horizontal bar in one script.*

```
import matplotlib.pyplot as plt

months    = ['Jan', 'Feb', 'Mar', 'Apr', 'May']
revenue   = [100, 140, 190, 210, 260]
students  = [1, 2, 3, 4, 5]
scores    = [55, 72, 80, 68, 95]
subjects  = ['Maths', 'Science', 'English']
sub_marks = [88, 92, 78]

# Line
plt.plot(months, revenue); plt.title('Revenue Trend'); plt.show()

# Bar
plt.bar(months, revenue, color='steelblue')
plt.title('Monthly Revenue'); plt.show()

# Pie
plt.pie(revenue, labels=months, autopct='%1.1f%%')
plt.title('Revenue Share'); plt.show()

# Scatter
plt.scatter(students, scores, color='crimson')
plt.title('Student Scores'); plt.show()

# Histogram
plt.hist(scores, bins=5, color='teal')
plt.title('Score Distribution'); plt.show()

# Horizontal bar
plt.barh(subjects, sub_marks, color='orchid')
plt.title('Subject Marks'); plt.show()
```

All programs in this document are functionally equivalent to the originals. Variable names, logic style and formatting have been intentionally changed for learning purposes.